SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

NAME : CHENNURU GODHASRI

REG. NO : 11249A060

CLASS : II B.E. (CSE)

SECTION : S – 4

COURSE CODE: BCSF244AI2

COURSE NAME: DATA SCIENCE TOOL KIT LAB

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(UNIVERSITY ESTABLISHED UNDER SECTION 3 OF UGC ACT 1956)
ENATHUR, KANCHIPURAM – 631 561

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this is the bonafide record of work done by **Mr/Ms. Chennuru Godhasri** ,

with **Reg. No 11249A060**_____of **II-B.E.(CSE)** during the academic year 2025 – 2026.

Course Code: BCSF244AI2

Course Name: DATA SCIENCE TOOL KIT LAB

Station: Enathur

Date:

Staff-in-charge **Head of the Department**

Submitted for the Practical examination held on_____.

Examiner-1 **Examiner-2**

INDEX

SL.NO	Name of the experiment	Date	Page no	Signature
1.	Introduction to python libraries			
1(A).	NumPy	19/01/26		
1(B).	Pandas	02/02/26		
1(C).	Mataplotib	02/02/26		
1(D).	Scikit-learn	14/02/26		
2.	Perform data exploration and preprocessing Python	14/02/26		
3.	Implement Regularized Linear Regression	14/02/26		
4.	Implement Naïve Bayes classifier	02/03/26		
5.	Implement Regularized Logistic regression	02/03/26		
6.	Bulid models using ensembling techniques	09/03/26		
7.	Bulid models using decision tree	23/03/26		
8.	Support vector machine with different Kernels	23/03/26		
9.	Implement K-Nearest Neighbours Classification	13/04/26		
10.	K-Means clustering with PCA and elbow Method	13/04/26		

Experiment 1:- Introduction to python libraries

1.1 NumPy

Aim: To implement array creation and operations using NumPy.

1. Array Creation

```python
import numpy as np

a = np.array([1,2,3,4,5])
b = np.arange(1,10,2)
c = np.linspace(0,1,5)
d = np.zeros((2,3))
e = np.ones((2,2))
f = np.eye(3)
g = np.random.randint(1, 100, size=(3,3))
print("array a:",a)
print("arange b:",b)
print("linspace c:",c)
print("zeros d:\n",d)
print("ones e:\n",e)
print("identity f:\n",f)
print("random_randint g:\n",g)
```

```
array a: [1 2 3 4 5]
arange b: [1 3 5 7 9]
linspace c: [0.   0.25 0.5  0.75 1.  ]
zeros d:
 [[0. 0. 0.]
```

2. Array Properties

```python
print("\nshape:",e.shape)
print("dimensions:",e.ndim)
print("size:",e.size)
print("data type:",e.dtype)
```

```
shape: (2, 2)
dimensions: 2
size: 4
data type: float64
```

```python
print("\nshape:",g.shape)
print("dimensions:",g.ndim)
print("size:",g.size)
print("data type:",g.dtype)
```

```
shape: (3, 3)
dimensions: 2
size: 9
data type: int64
```

3. Reshape and Transpose

Reshape&transpose

```
Reshaped = g.reshape(1,9)
print("Reshaped array:\n",Reshaped)
print("Transpose:\n",g.T)
print("Flattered:",g.flatten())
```

```
Reshaped array:
 [[66 22 31 56 45 17 26 80 18]]
Transpose:
 [[66 56 26]
 [22 45 80]
 [31 17 18]]
Flattered: [66 22 31 56 45 17 26 80 18]
```

4. Mathematical Operations

Mathematical operations

```
a = np.array([1,2,3,4,5])
print("\nAdd:",np.add(a,2))
print("Multiply:",np.multiply(a,3))
print("Square Root:",np.sqrt(a))
print("Power",np.power(a,2))
print("Absoulte:",np.abs([-1,-2,-3]))
```

```
Add: [3 4 5 6 7]
Multiply: [ 3  6  9 12 15]
Square Root: [1.         1.41421356 1.73205081 2.         2.23606798]
Power [ 1  4  9 16 25]
Absoulte: [1 2 3]
```

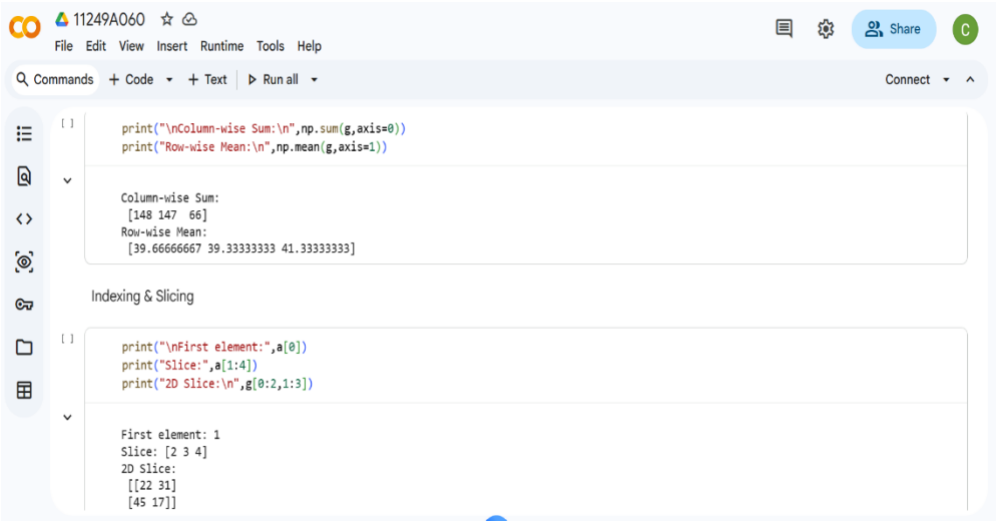
5. Statistical Functions

Statistical functions

```
a = np.array([1,2,3,4,5])
print("\nMean:",np.mean(a))
print("Median:",np.median(a))
print("Standard Deviation:",np.std(a))
print("Variance:",np.var(a))
print("Minimum:",np.min(a))
print("Maximum:",np.max(a))
print("sum:",np.sum(a))
print("Percentile (50):",np.percentile(a,50))
```

```
Mean: 3.0
Median: 3.0
Standard Deviation: 1.4142135623730951
Variance: 2.0
Minimum: 1
Maximum: 5
sum: 15
Percentile (50): 3.0
```

6. Axis based Operations



```
print("\nColumn-wise Sum:\n",np.sum(g,axis=0))
print("Row-wise Mean:\n",np.mean(g,axis=1))
```

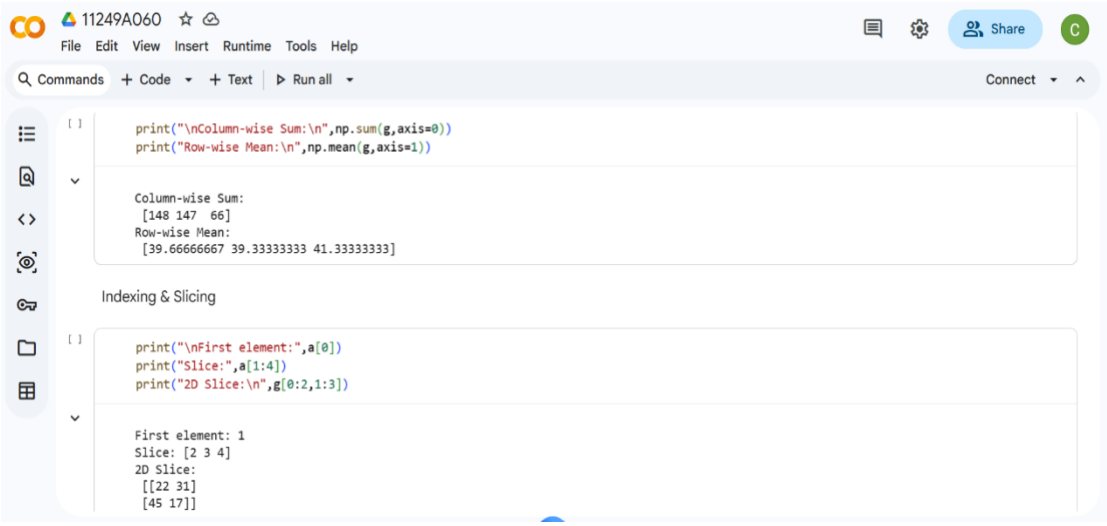
```
Column-wise Sum:
 [148 147  66]
Row-wise Mean:
 [39.66666667 39.33333333 41.33333333]
```

Indexing & Slicing

```
print("\nFirst element:",a[0])
print("Slice:",a[1:4])
print("2D Slice:\n",g[0:2,1:3])
```

```
First element: 1
Slice: [2 3 4]
2D Slice:
 [[22 31]
 [45 17]]
```

7. Indexing & Slicing



```
print("\nColumn-wise Sum:\n",np.sum(g,axis=0))
print("Row-wise Mean:\n",np.mean(g,axis=1))
```

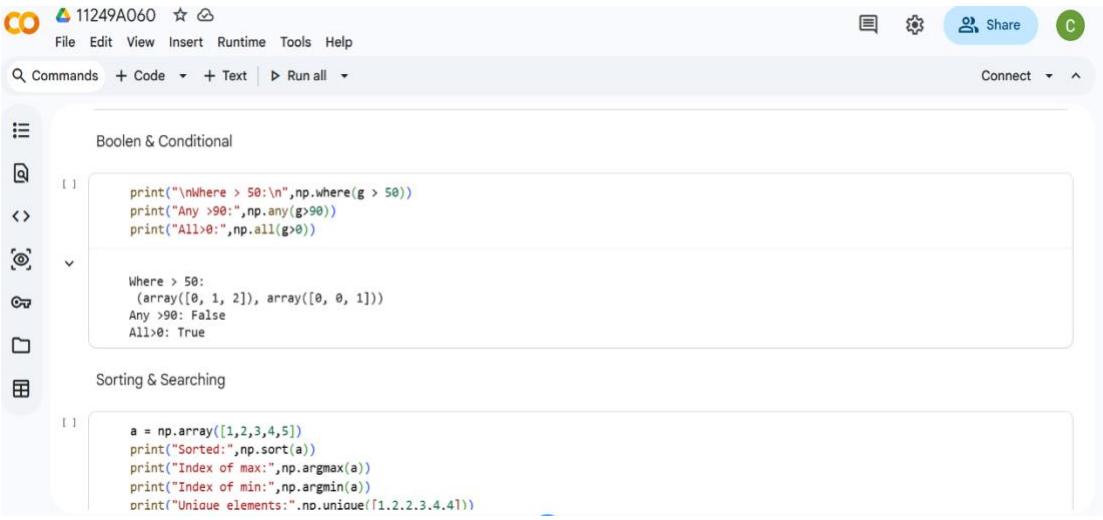
```
Column-wise Sum:
 [148 147  66]
Row-wise Mean:
 [39.66666667 39.33333333 41.33333333]
```

Indexing & Slicing

```
print("\nFirst element:",a[0])
print("Slice:",a[1:4])
print("2D Slice:\n",g[0:2,1:3])
```

```
First element: 1
Slice: [2 3 4]
2D Slice:
 [[22 31]
 [45 17]]
```

8. Boolean and Conditional



Boolen & Conditional

```
print("\nWhere > 50:\n",np.where(g > 50))
print("Any >90:",np.any(g>90))
print("All>0:",np.all(g>0))
```

```
Where > 50:
 (array([0, 1, 2]), array([0, 0, 1]))
Any >90: False
All>0: True
```

Sorting & Searching

```
a = np.array([1,2,3,4,5])
print("Sorted:",np.sort(a))
print("Index of max:",np.argmax(a))
print("Index of min:",np.argmin(a))
print("Unique elements:",np.unique([1,2,2,3,4,4]))
```

9. Sorting and Searching

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

Boolen & Conditional

```
print("\nWhere > 50:\n",np.where(g > 50))
print("Any >90:",np.any(g>90))
print("All>0:",np.all(g>0))
```

```
Where > 50:
 (array([0, 1, 2]), array([0, 0, 1]))
Any >90: False
All>0: True
```

Sorting & Searching

```
a = np.array([1,2,3,4,5])
print("Sorted:",np.sort(a))
print("Index of max:",np.argmax(a))
print("Index of min:",np.argmin(a))
print("Unique elements:",np.unique([1,2,2,3,4,4]))
```

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

```
print("Sorted:",np.sort(a))
print("Index of max:",np.argmax(a))
print("Index of min:",np.argmin(a))
print("Unique elements:",np.unique([1,2,2,3,4,4]))
```

```
Sorted: [1 2 3 4 5]
Index of max: 4
Index of min: 0
Unique elements: [1 2 3 4]
```

Linear Algebra

```
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])
print("\nDot product:\n",np.dot(x,y))
```

```
Dot product:
 [[19 22]
 [43 50]]
```

10. Linear Algebra

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

```
print("Sorted:",np.sort(a))
print("Index of max:",np.argmax(a))
print("Index of min:",np.argmin(a))
print("Unique elements:",np.unique([1,2,2,3,4,4]))
```

```
Sorted: [1 2 3 4 5]
Index of max: 4
Index of min: 0
Unique elements: [1 2 3 4]
```

Linear Algebra

```
x = np.array([[1,2],[3,4]])
y = np.array([[5,6],[7,8]])
print("\nDot product:\n",np.dot(x,y))
```

```
Dot product:
 [[19 22]
 [43 50]]
```

Result: Array operations using NumPy were performed successfully.

1.2 Pandas (Python Data Analysis Library)

Aim: To implement data creations, filtering, and cleaning using Pandas.

A. Creating Data Frames

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

A.creating dataframes

```
import pandas as pd
data={
    'Name':['alice','Bob','Charlie'],
    'Age':[25,30,35],
    'City':['New york','Paris','London']
}
df=pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
```

```
      Name  Age      City
0    alice   25  New york
1      Bob   30     Paris
2  Charlie   35    London
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
```

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

```
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
             Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
```

B.selection and filtering

```
import pandas as pd
data={
    'Name':['alice','Bob','Charlie'],
```

B. Selection and Filtering

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

```
---  ------  --------------  -----
 0   Name    3 non-null      object
 1   Age     3 non-null      int64
 2   City    3 non-null      object
dtypes: int64(1), object(2)
memory usage: 204.0+ bytes
None
             Age
count   3.0
mean   30.0
std     5.0
min    25.0
25%    27.5
50%    30.0
75%    32.5
max    35.0
```

B.selection and filtering

```
import pandas as pd
data={
    'Name':['alice','Bob','Charlie'],
```

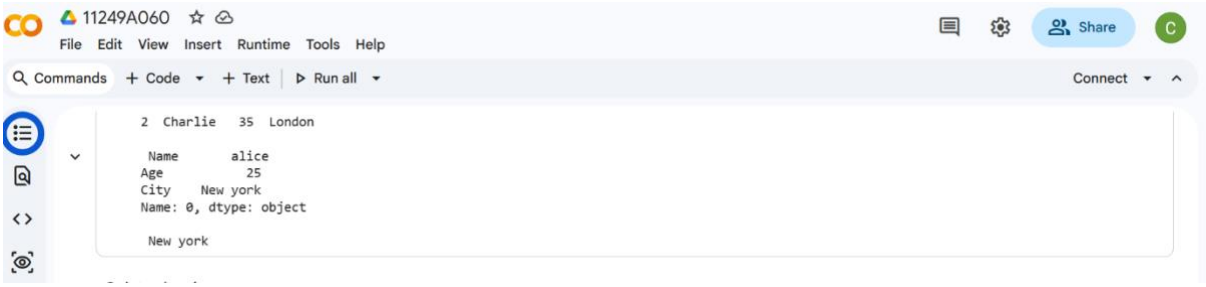
```
    'Age':[25,30,35],
    'City':['New york','Paris','London']
}
df=pd.DataFrame(data)

ages = df['Age']
above_25 = df[df['Age'] > 25]
row_0 = df.iloc[0]
specific_val = df.loc[0, 'City']

print(ages,"\n\n",above_25,"\n\n",row_0,"\n\n",specific_val)
```

```
0    25
1    30
2    35
Name: Age, dtype: int64

       Name  Age    City
1       Bob   30   Paris
2   Charlie   35  London
```

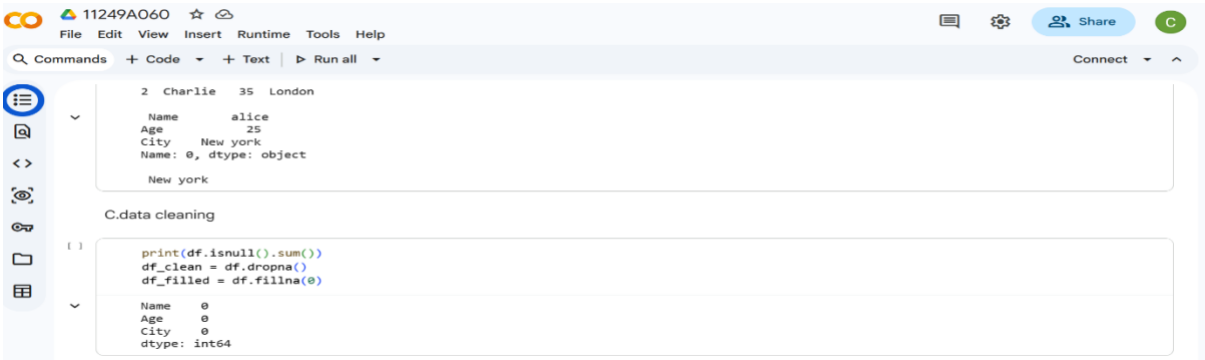


```
2  Charlie   35  London

 Name       alice
Age           25
City     New york
Name: 0, dtype: object

 New york
```

B. Data Cleaning



```
2  Charlie   35  London

 Name       alice
Age           25
City     New york
Name: 0, dtype: object

 New york
```

C.data cleaning

```
print(df.isnull().sum())
df_clean = df.dropna()
df_filled = df.fillna(0)
```

```
Name    0
Age     0
City    0
dtype: int64
```

Result: Pandas operations were performed successfully.

1.3 MATPLOTLIB (Plotting Library)

Aim: To implement plotting operations using Matplolib.

Basic plot program

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data = {
    'StudyHours': [1,2,3,4,5,6,7,8],
    'ExamScorce': [35,40,50,55,65,70,78,85]
}
df = pd.DataFrame(data)
print(df.head())
print(df.info())
print(df.describe())
```

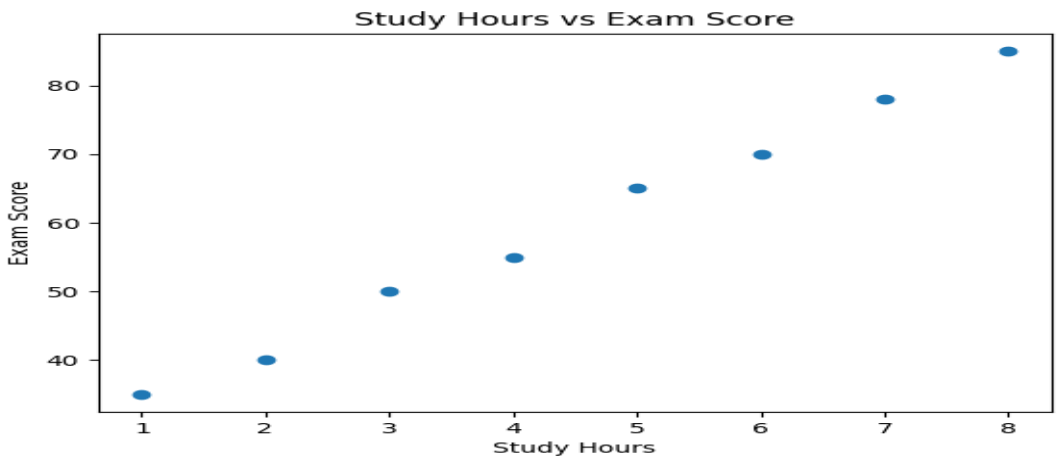
```
   StudyHours  ExamScorce
0           1          35
1           2          40
2           3          50
3           4          55
4           5          65
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8 entries, 0 to 7
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   StudyHours  8 non-null      int64
 1   ExamScorce  8 non-null      int64
dtypes: int64(2)
memory usage: 260.0 bytes
None
       StudyHours  ExamScorce
count     8.00000    8.000000
mean      4.50000   59.750000
```

1. Scatter plot

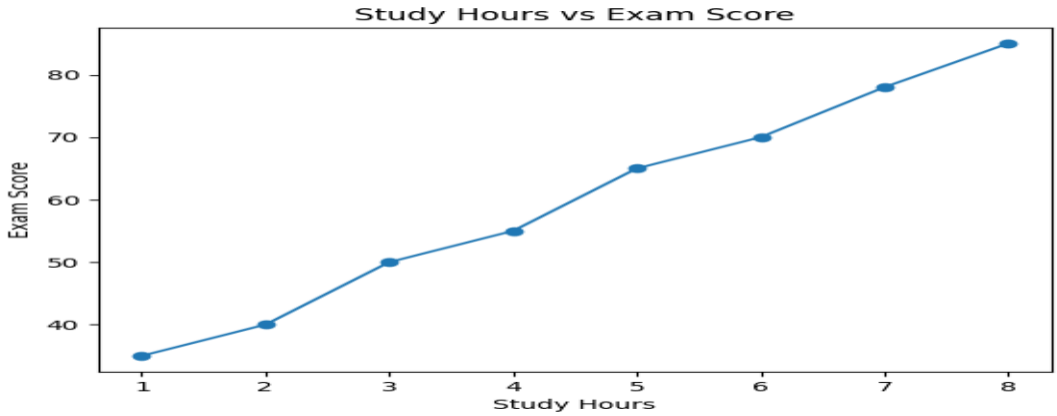
```
       std       2.44949   17.822538
       min       1.00000   35.000000
       25%       2.75000   47.500000
       50%       4.50000   60.000000
       75%       6.25000   72.000000
       max       8.00000   85.000000
```

1 SCATTER PLOT

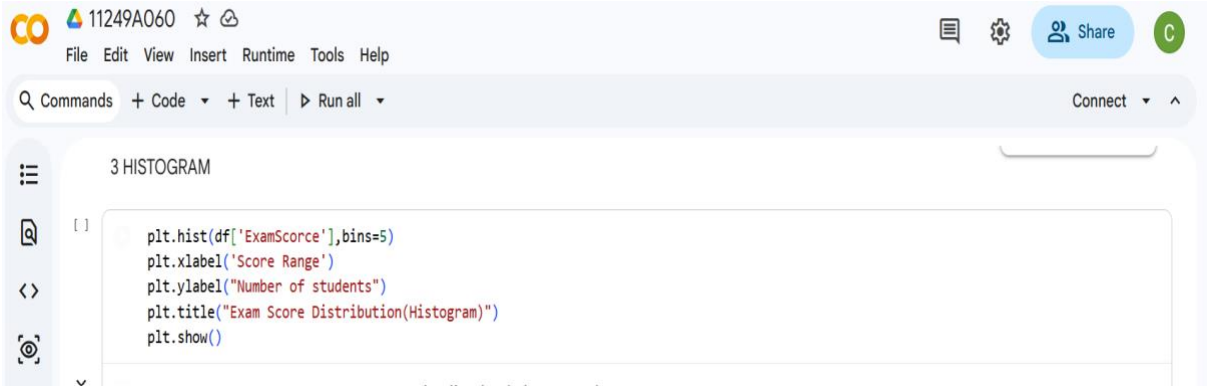
```
plt.scatter(df['StudyHours'],df['ExamScorce'])
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Study Hours vs Exam Score')
plt.show()
```

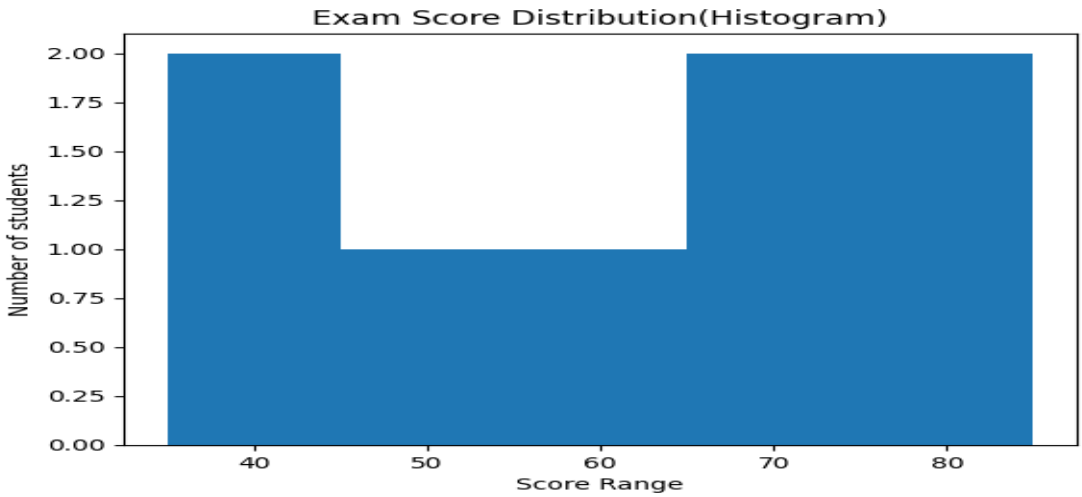


2. Line plot



3. Histogram





4. Bar Chart

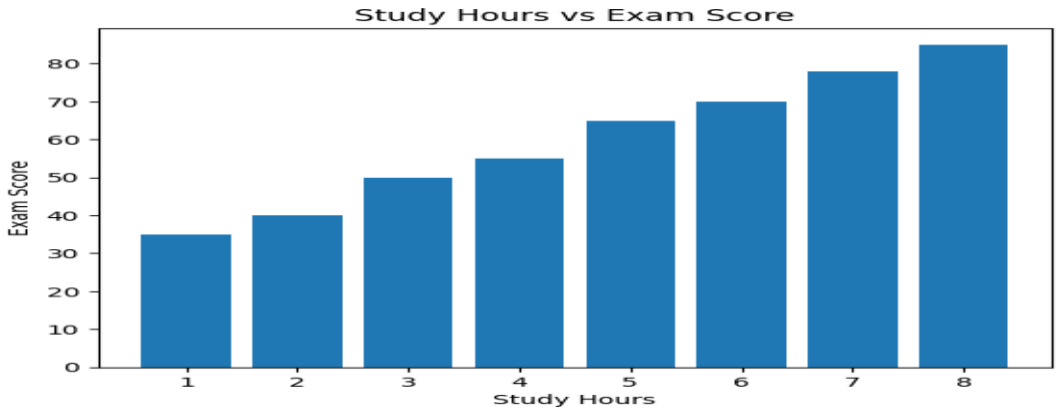
11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

BAR CHART

```
plt.bar (df['StudyHours'], df['ExamScorce'])
plt.xlabel('Study Hours')
plt.ylabel('Exam Score')
plt.title('Study Hours vs Exam Score')
plt.show()
```



Result: Plots were generated successfully.

1.4 Scikit learn (Machine Learning)

Aim: To implement machine learning algorithms using Scikit-learn.

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

SCIKIT-LEARN

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
#step2
data = {
    'Student':['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
    'Classes_Attended':[30,35,40,45,50,55,60,65,70,75],
    'Internal_Marks':[35,38,42,46,50,55,60,65,70,75]
}
df = pd.DataFrame(data)
print(df)

print("**********************")
#step3
X = df[['Classes_Attended']]
y = df['Internal_Marks']
```

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

```python
#step4
X_train, X_test, y_train, y_Test = train_test_split(X, y, test_size=0.2, random_state=7)

#step5
model = LinearRegression()
model.fit(X_train, y_train)

#step6
predictions = model.predict(X_test)

#step7
mse = mean_squared_error(y_Test, predictions)
print("Mean Squared Error:", mse)
print("MSE:",mse)
print("Marks per Class:",model.coef_[0])
print("Baase Marks:",model.intercept_)

 #step8

plt.scatter(X,y)
plt.plot(X, model.predict(X))
```

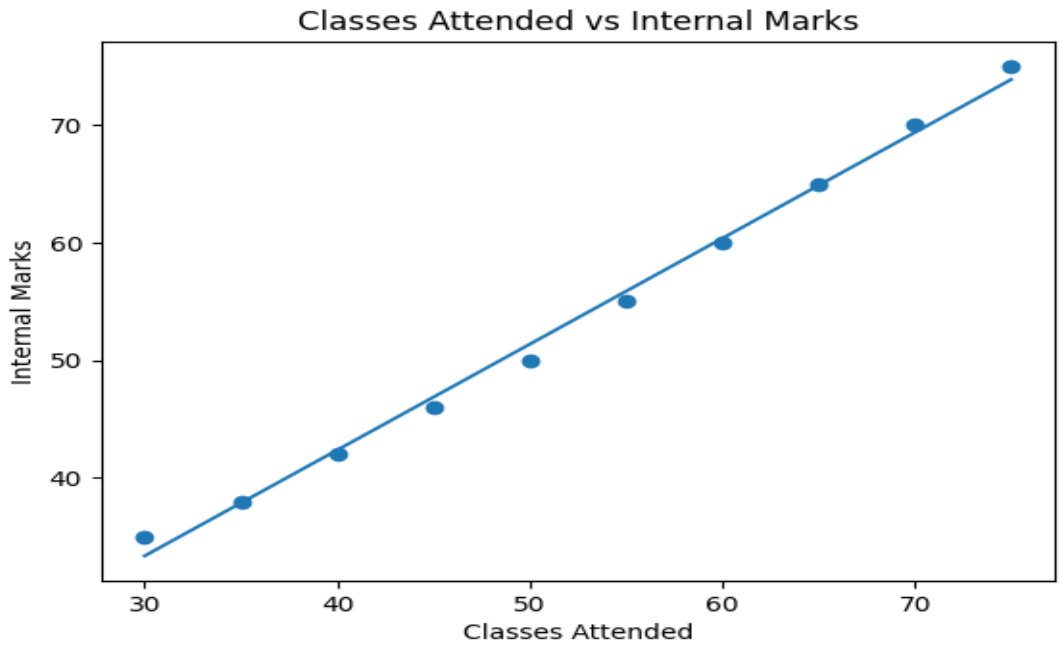
11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all Connect Share

```python
plt.xlabel('Classes Attended')
plt.ylabel('Internal Marks')
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

```
  Student  Classes_Attended  Internal_Marks
0      S1                30              35
1      S2                35              38
2      S3                40              42
3      S4                45              46
4      S5                50              50
5      S6                55              55
6      S7                60              60
7      S8                65              65
8      S9                70              70
9     S10                75              75
**********************
Mean Squared Error: 0.578125
MSE: 0.578125
Marks per Class: 0.9
Baase Marks: 6.375
```



Result: Machine learning models were implemented successfully.

Experiment 2:-Perform Data Exploration and preprocessing in python

Aim: To implement data exploration and preprocessing.

11249a060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

EXP NO:2

PERFORM DATA EXPLORATION & PREPOCESSING

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

# -- Step 1: Load Data
data = {
    'Country': ['India', 'USA', 'India', 'USA', 'UK', 'India'],
    'Age': [22, 25, np.nan, 30, 28, 35],
    'Salary': [40000, 60000, 50000, np.nan, 72000, 58000],
    'Purchased': ['No', 'Yes', 'No', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)

print("--- ORIGINAL RAW DATA ---")
print(df)
print("\n")

# -- Step 2: Inspect Data
print("--- MISSING VALUES COUNT ---")
print(df.isnull().sum())
print("\n")

# -- Step 3: Clean Data
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Salary'] = df['Salary'].fillna(df['Salary'].mean())

print("--- DATA AFTER CLEANING ---")
print(df)
print("\n")

# -- Step 4: Convert Text to Numbers
```

11249a060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
print("\n")

# -- Step 4: Convert Text to Numbers
df_encoded = pd.get_dummies(df, columns=['Country'])
df_encoded['Purchased'] = df_encoded['Purchased'].map({'No': 0, 'Yes': 1})

print("--- DATA AFTER ENCODING (All Numbers Now) ---")
print(df_encoded)
print("\n")

# -- Step 5: Scale Features
X = df_encoded.drop('Purchased', axis=1)
y = df_encoded['Purchased']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

print("--- FINAL PROCESSED DATA (Ready for AI) ---")
print(pd.DataFrame(X_scaled, columns=X.columns).head())
```

```
--- ORIGINAL RAW DATA ---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   India   NaN  50000.0        No
3     USA  30.0      NaN       Yes
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0        No


--- MISSING VALUES COUNT ---
Country      0
Age          1
Salary       1
Purchased    0
dtype: int64


--- DATA AFTER CLEANING ---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
```

```
3     USA  30.0      NaN       Yes
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0        No


--- MISSING VALUES COUNT ---
Country      0
Age          1
Salary       1
Purchased    0
dtype: int64


--- DATA AFTER CLEANING ---
  Country   Age   Salary Purchased
0   India  22.0  40000.0        No
1     USA  25.0  60000.0       Yes
2   India  28.0  50000.0        No
3     USA  30.0  56000.0       Yes
4      UK  28.0  72000.0       Yes
5   India  35.0  58000.0        No


--- DATA AFTER ENCODING (All Numbers Now) ---
    Age   Salary  Purchased  Country_India  Country_UK  Country_USA
0  22.0  40000.0          0           True       False        False
1  25.0  60000.0          1          False       False         True
2  28.0  50000.0          0           True       False        False
3  30.0  56000.0          1          False       False         True
4  28.0  72000.0          1          False        True        False
5  35.0  58000.0          0           True       False        False


--- FINAL PROCESSED DATA (Ready for AI) ---
        Age    Salary  Country_India  Country_UK  Country_USA
0 -1.484615 -1.644453            1.0   -0.447214    -0.707107
1 -0.742307  0.411113           -1.0   -0.447214     1.414214
2  0.000000 -0.616670            1.0   -0.447214    -0.707107
3  0.494872  0.000000           -1.0   -0.447214     1.414214
4  0.000000  1.644453           -1.0    2.236068    -0.707107
```

Result: Data preprocessing was performed successfully.

Experiment 3:-Implement Regularized Linear Regression

Aim: To implement regularized linear regression.

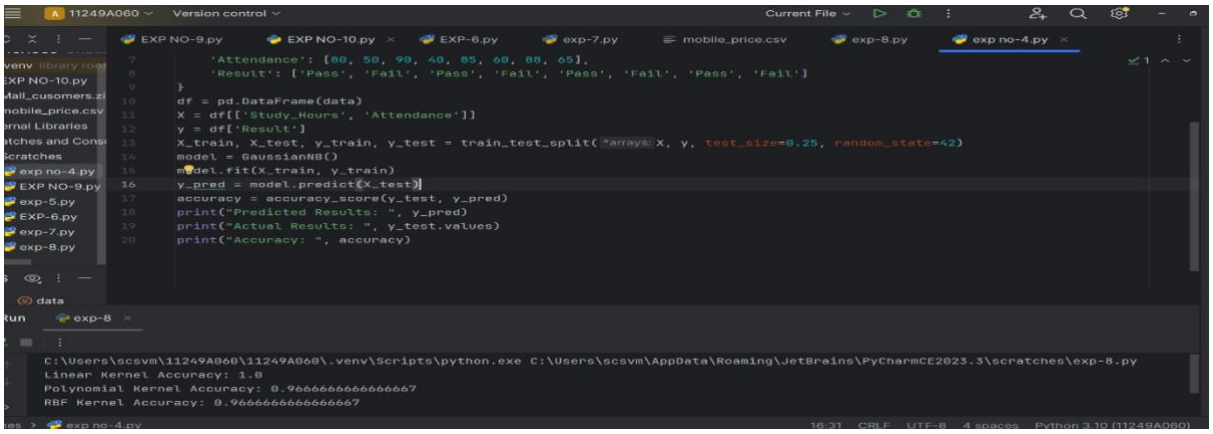
```
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn import datasets
df = pd.read_csv('/content/diabets.zip')
diabetes = datasets.load_diabetes()
X_train, X_test, y_train, y_test = train_test_split(diabetes.data,
diabetes.target, test_size=0.2, random_state=42)
lr = LinearRegression().fit(X_train, y_train)
ridge = Ridge (alpha=1.0).fit(X_train, y_train)
lasso = Lasso (alpha=0.1).fit(X_train, y_train)
print (f"Linear Regression MSE: {mean_squared_error(y_test,lr.predict(X_test)):.2f}")
print (f"Ridge Regression MSE: {mean_squared_error(y_test,ridge.predict(X_test)):.2f}")
print (f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(X_test)):.2f}")
```

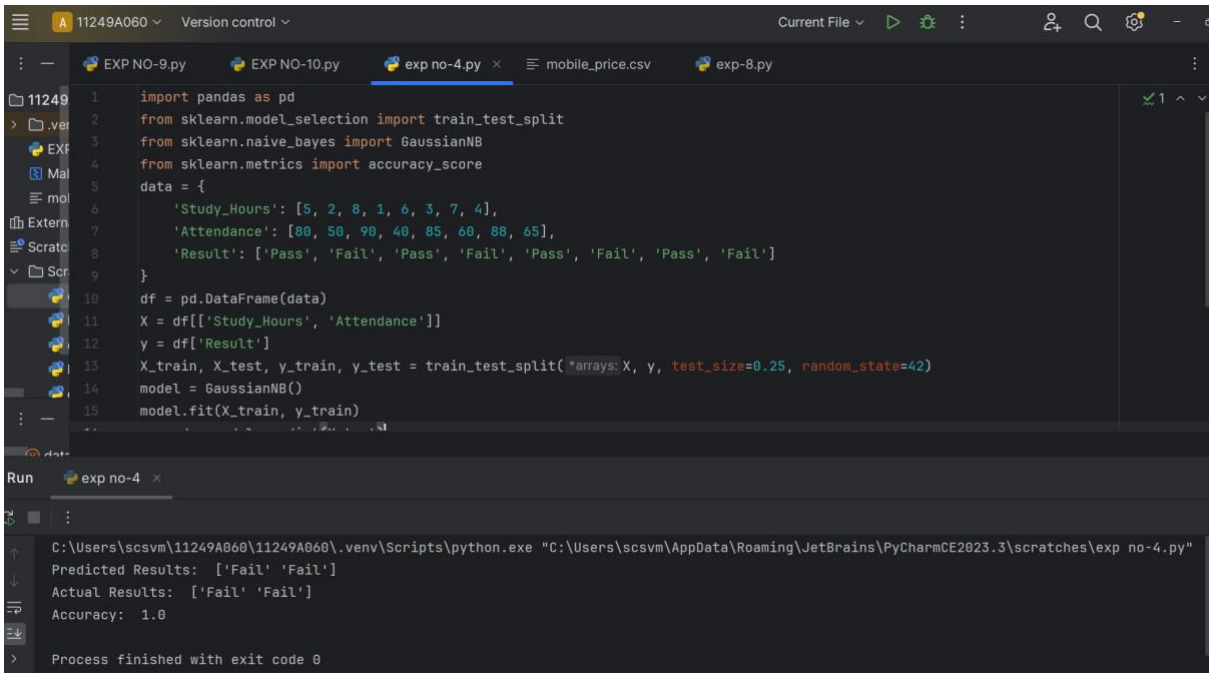
```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 2798.19
```

Result: The model was implemented successfully.

Experiment 4:-Implement Naive Bayes Classifier

Aim: To implement Navie Bayes Classification.





Result: The Classifier was implemented successfully.

Experiment 5:-Implement Regularised logistic regression

Aim: To implement regularized logistic regression.

```python

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test,y_train,y_test = train_test_split(*arrays: X,y,test_size=0.2,random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

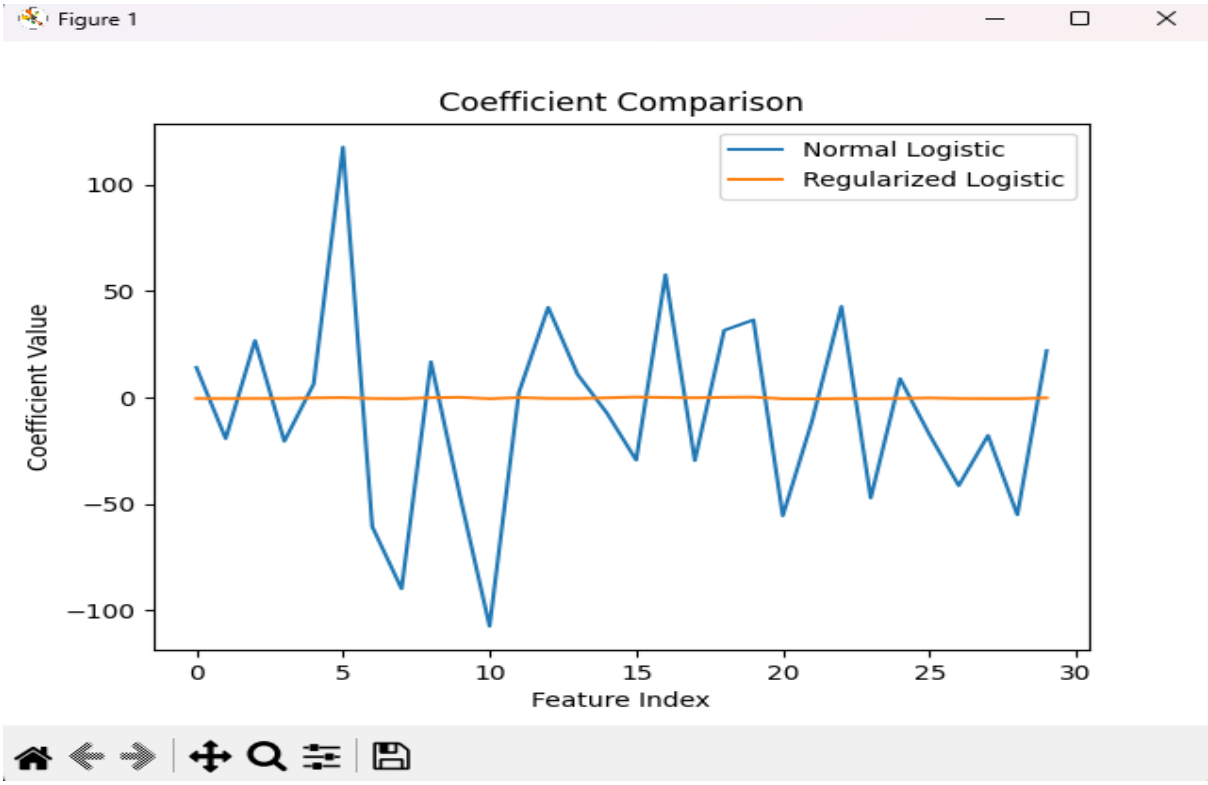
```
C:\Users\scsvm\11249A060\11249A060\.venv\Scripts\python.exe C:\Users\scsvm\AppData\Roaming\JetBrains\PyCharmCE2023.3\scratches\exp-5.py
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 99.12280701754386
```

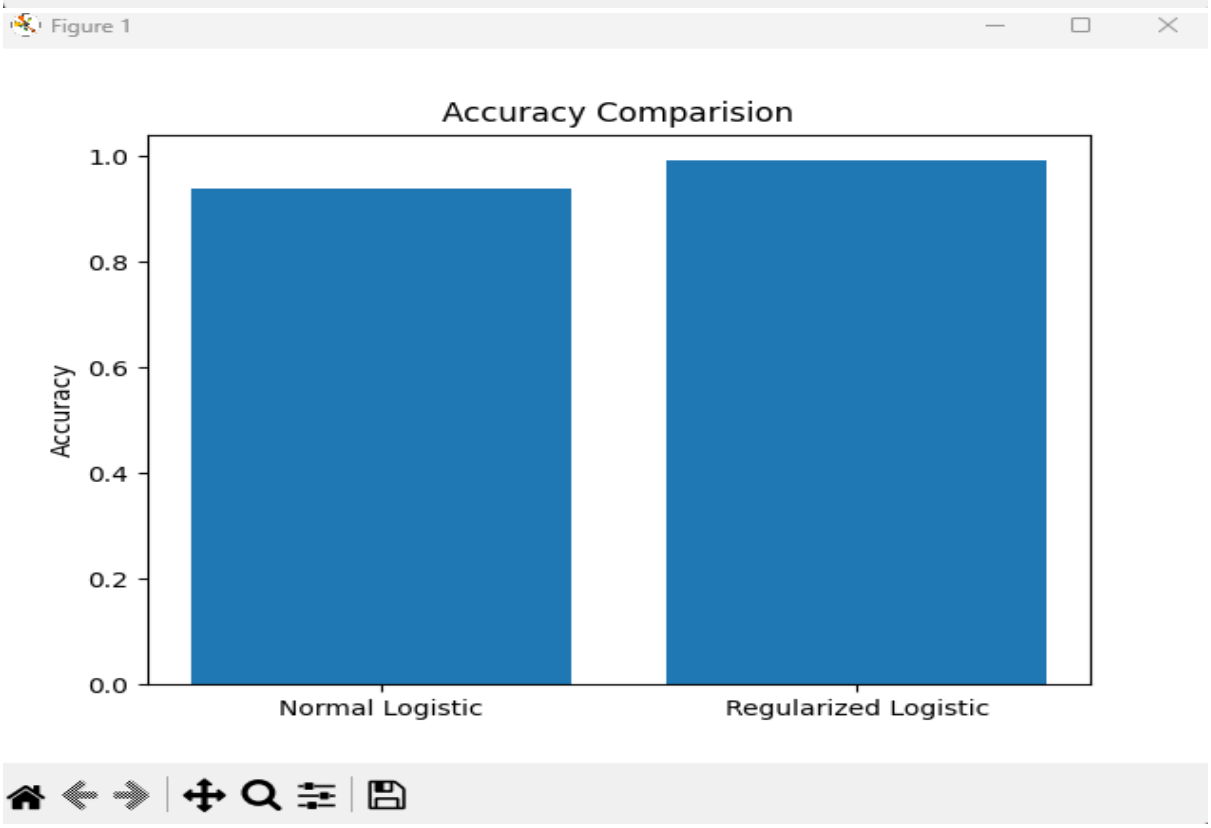
```python
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear')
normal_model.fit(X_train_scaled, y_train)
normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)

regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear')
regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)

print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)

plt.figure()
plt.plot(*args: normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot(*args: regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
```

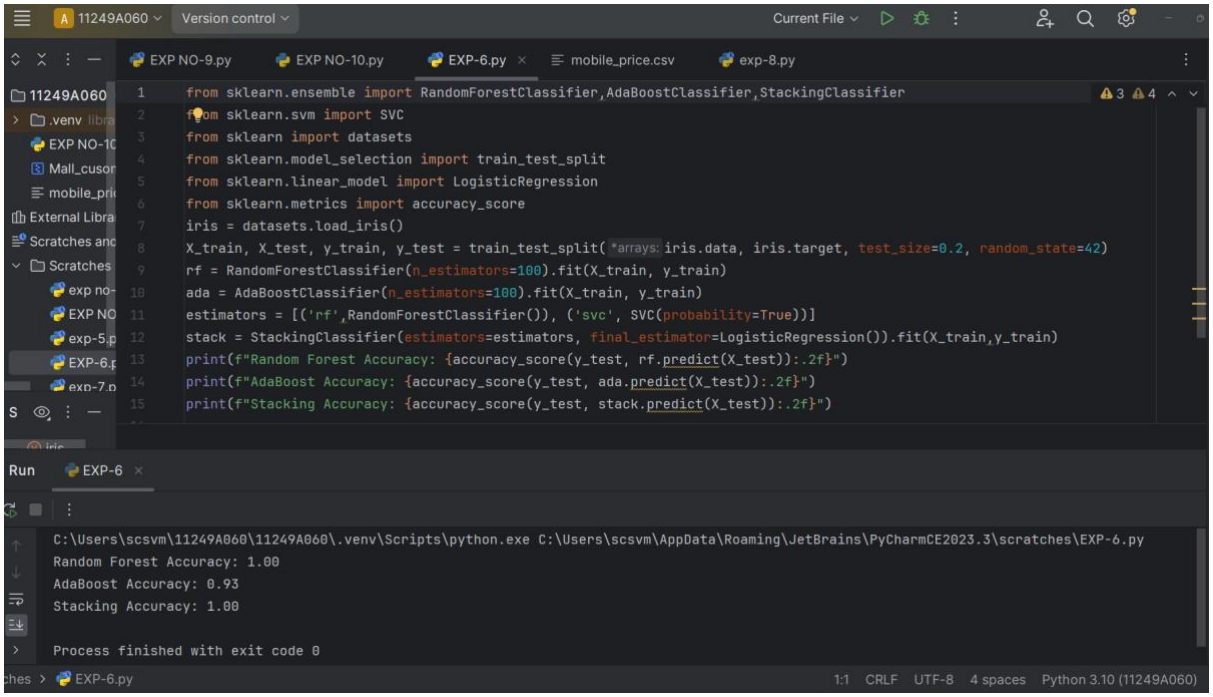




Result: The model was implemented successfully.

Experiment 6:-Build Models using different Ensembling Techniques

Aim: To implement ensembling techniques.



Result: The model was implemented successfully.

Experiment 7:-Build Models using Decision Trees

Aim: To implement decision tree models.

```python
from sklearn. tree import DecisionTreeClassifier
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split( *arrays: iris.data, iris.target,test_size=0.2, random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3).fit(X_train, y_train)
print(f"Decision Tree Classifier Accuracy: {accuracy_score(y_test,dt_clf.predict(X_test)):.2f}")
print("Feature Importances:",dt_clf.feature_importances_)
```

```
C:\Users\scsvm\11249A060\11249A060\.venv\Scripts\python.exe C:\Users\scsvm\AppData\Roaming\JetBrains\PyCharmCE2023.3\scratches\exp-7.py
Decision Tree Classifier Accuracy: 1.00
Feature Importances: [0.         0.         0.93462632 0.06537368]

Process finished with exit code 0
```

Result: The model was implemented successfully.

Experiment 8: Support Vector Machines (SVM) with various Kernels

Aim: To implement Support Vector Machine.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
# Load dataset
data = load_iris()
X = data.data
y = data.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2)
# Linear Kernel
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
```

```
C:\Users\scsvm\11249A060\11249A060\.venv\Scripts\python.exe C:\Users\scsvm\AppData\Roaming\JetBrains\PyCharmCE2023.3\scratches\exp-8.py
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9666666666666667

Process finished with exit code 0
```

```python
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)

# Polynomial Kernel
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)
# RBF Kernel
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

```
C:\Users\scsvm\11249A060\11249A060\.venv\Scripts\python.exe C:\Users\scsvm\AppData\Roaming\JetBrains\PyCharmCE2023.3\scratches\exp-8.py
Linear Kernel Accuracy: 1.0
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9666666666666667
```

Result: The model was implemented successfully.

Experiment 9:-Implement KNN Classification

Aim: To implement K-Nearest Neighbors algorithm.

```python
# Step 1: Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Step 2: Load dataset
data = pd.read_csv("/content/mobile_price.csv")

# Step 3: Define features and target
X = data.drop ("blue", axis=1)
y = data["blue"]

# Step 4: Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 5: Feature scaling
scaler = StandardScaler()
```

```
)

# Step 5: Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Step 6: Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Step 7: Train model
knn.fit(X_train, y_train)

# Step 8: Prediction
y_pred = knn.predict(X_test)

# Step 9: Evaluation
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.465
Confusion Matrix:
 [[39 56]
 [51 54]]
Classification Report:
              precision    recall  f1-score   support

           0       0.43      0.41      0.42        95
           1       0.49      0.51      0.50       105

    accuracy                           0.47       200
   macro avg       0.46      0.46      0.46       200
weighted avg       0.46      0.47      0.46       200
```

Result: The model was implemented successfully.

Experiment 10:-K- Means Clustering with PCA and Elbow Method

Aim: To implement k-means clustering.

11249a060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

EXP-10

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans

data = pd.read_csv('/content/Mall_Customers.csv')
x = data[['Age','Annual Income (k$)','Spending Score (1-100)']]
scaler = StandardScaler()
x_scaled = scaler.fit_transform(x)
pca = PCA(n_components=2)
x_pca = pca.fit_transform(x_scaled)
print("Explaned variance Ratio:",pca.explained_variance_ratio_)
wcss = []

for i in range(1,11):
    kmeans = KMeans(n_clusters=i,random_state=0)
    kmeans.fit(x_pca)
```

11249a060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
    kmeans.fit(x_pca)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,11),wcss,marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters(k)")
plt.ylabel("WCSS")
plt.show()

kmeans = KMeans(n_clusters=5,random_state=0)
cluster = kmeans.fit_predict(x_pca)

plt.scatter(x_pca[:,0],x_pca[:,1],c=cluster)
plt.title("k_Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA component 2")
plt.show()
```

```
Explaned variance Ratio: [0.44266167 0.33308378]
```

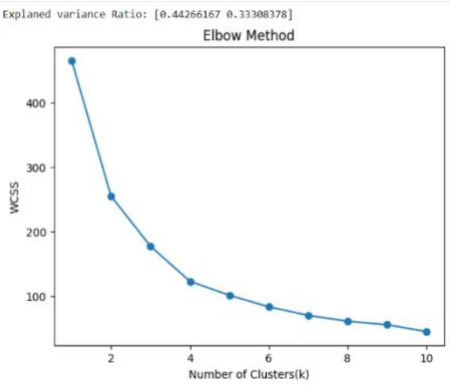
11249a060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

```
Explaned variance Ratio: [0.44266167 0.33308378]
```



Result: Clustering was performed successfully.

STUDY CASE :

Aim: To Create a file students.csv contains details

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
       reg no   Name Dept Section  age
0  11249023    sri  cse     S-4   19
1  11246778    ABC  CSE     S-4   19
2   1122345  QWERE  CSE     S-4   19
3   1124924   ASDD  CSE     S-4   18
4   1124925   ZXCV  CSE     S-4   20
5  11249023    sri  cse     S-4   19
6  11246778    ABC  CSE     S-4   19
7   1122345  QWERE  CSE     S-4   19
8   1124924   ASDD  CSE     S-4   18
9   1124925   ZXCV  CSE     S-4   20
       reg no   Name Dept Section  age
0  11249023    sri  cse     S-4   19
1  11246778    ABC  CSE     S-4   19
2   1122345  QWERE  CSE     S-4   19
3   1124924   ASDD  CSE     S-4   18
4   1124925   ZXCV  CSE     S-4   20
     Name  age
0     sri   19
1     ABC   19
2   QWERE   19
3    ASDD   18
4    ZXCV   20
```

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
3   ASDD   18
4   ZXCV   20
5    sri   19
6    ABC   19
7  QWERE   19
8   ASDD   18
9   ZXCV   20
people with age above 10
       reg no   Name Dept Section  age
0  11249023    sri  cse     S-4   19
1  11246778    ABC  CSE     S-4   19
2   1122345  QWERE  CSE     S-4   19
3   1124924   ASDD  CSE     S-4   18
4   1124925   ZXCV  CSE     S-4   20
5  11249023    sri  cse     S-4   19
6  11246778    ABC  CSE     S-4   19
7   1122345  QWERE  CSE     S-4   19
8   1124924   ASDD  CSE     S-4   18
9   1124925   ZXCV  CSE     S-4   20
********************
age
19    6
18    2
20    2
```

11249A060

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
1  11246778    ABC  CSE     S-4   19
2   1122345  QWERE  CSE     S-4   19
3   1124924   ASDD  CSE     S-4   18
4   1124925   ZXCV  CSE     S-4   20
5  11249023    sri  cse     S-4   19
6  11246778    ABC  CSE     S-4   19
7   1122345  QWERE  CSE     S-4   19
8   1124924   ASDD  CSE     S-4   18
9   1124925   ZXCV  CSE     S-4   20
********************
age
19    6
18    2
20    2
Name: count, dtype: int64
19.0
************************
       reg no   Name Dept Section  age
0  11249023    sri  cse     S-4   19
1  11246778    ABC  CSE     S-4   19
2   1122345  QWERE  CSE     S-4   19
3   1124924   ASDD  CSE     S-4   18
4   1124925   ZXCV  CSE     S-4   20
```

Result: The program is executed successfully.